

LIBERTAS

Curso: Sistemas de Informação

Disciplina: Estrutura de Dados I

Professor: MSc. Jonathan Henrique Jeremias Souza

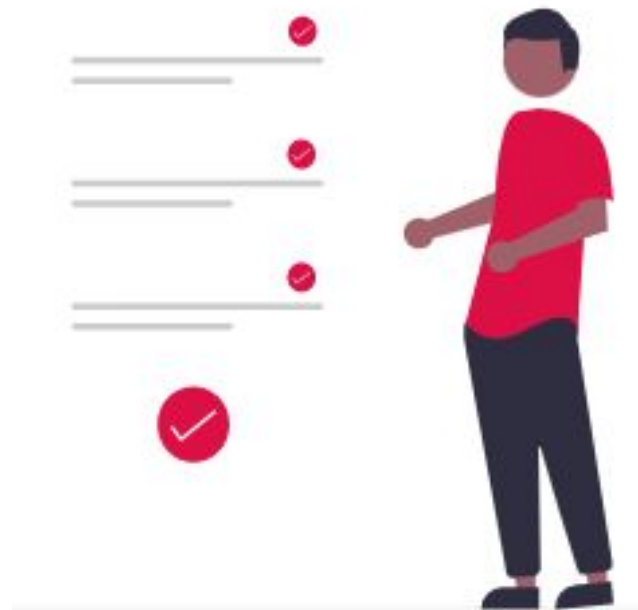
E-mail: jonathansouza@libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

AGENDA

- Revisão de conceitos;
- Tipos de Dados para Estrutura de Dados;



DEFINIÇÃO DE DADOS

Introdução

- De acordo com Setzer (2015), dado é uma "sequência de símbolos quantificados ou quantificáveis", e o estudo dos algoritmos tem como base os dados, pois, para que ocorra a resolução de um problema, faz-se necessária a escolha da representação da realidade, geralmente definida em termos de dados e suas representações.
- Os dados são valores fornecidos pelo usuário do programa, podendo ser obtidos a partir do processamento, arquivos, bancos de dados ou outros programas.

DEFINIÇÃO DE DADOS

Introdução

- Os dados são armazenados temporariamente em variáveis, para que sejam processados, de acordo com as especificações do algoritmo.
- Para garantir a integridade do resultado obtido, devem ser classificados de acordo com o tipo do valor a ser armazenado na variável, evitando problemas ocasionados pelo fornecimento de valores inadequados à operação realizada.

DEFINIÇÃO DE DADOS

Introdução

- Vamos supor que o algoritmo deva especificar os passos para efetuar a soma de dois números quaisquer, fornecidos pelo usuário.
- Então, as variáveis definidas para receber estes dados devem ser do tipo numérico, compatível com a operação que será realizada.

TIPOS DE DADOS

Introdução

- Os tipos de dados costumam ser definidos a partir dos tipos primitivos criados em função das características dos computadores.
- Como os dados manipulados pelos computadores durante a execução dos programas são armazenados na memória, esses tipos seguem as características de formato e estrutura definidas para essa memória.
- Assim, são organizados em bits e bytes e suas combinações.

TIPOS DE DADOS

Introdução

- Por exemplo, para representar um número inteiro, poderiam ser usados 2 bytes ou 16 bits.
- Isto resultaria em 2^{16} combinações possíveis para a representação de números, ou 65.536 possibilidades, considerando os estados possíveis que um bit pode assumir: 0 ou 1.
- Lembre-se de que os números poderiam assumir valores negativos e positivos; nesta faixa, teríamos representações que iriam de -32.768 a 32.767 .

TIPOS DE DADOS

Introdução

- Definir o tipo de dado mais adequado para ser armazenado em uma variável é importante para garantir a resolução do problema.
- Ao desenvolver um algoritmo, é necessário que se tenha conhecimento prévio do tipo de informação (dado) que será utilizado para resolver o problema proposto.
- A partir daí, escolhe-se o tipo de dado adequado para a variável que representará esse valor.

TIPOS DE DADOS

Tipos primitivos

- Tipos primitivos de dados são grupos de valores indivisíveis, como os tipos literal, inteiro, real e lógico, utilizados na construção de algoritmos, cujos equivalentes, em Java, são char, int, float e boolean.
- Uma variável do tipo boolean, por exemplo, pode assumir o valor "verdadeiro" ou o valor "falso" e nenhum outro.

TIPOS DE DADOS

Tipos primitivos

- Já na elaboração de programas, estes tipos de dados podem se desdobrar, com o objetivo de se adequarem melhor ao propósito de cada linguagem e à resolução prática dos problemas.
- A Tabela 3.1 apresenta, de forma resumida, a definição dos quatro tipos de dados primitivos, bem como seus equivalentes, na linguagem de programação Java, indicando também sua capacidade de armazenamento.

TIPOS DE DADOS

Tipos primitivos

Primitivos		Linguagem Java	
Tipo	Descrição	Tipo	Capacidade de armazenamento
<code>literal</code> (texto ou caractere)	Representa uma sequência de um ou mais caracteres (letras, números e símbolos); contudo, números são armazenados como um caractere, não podendo ser utilizados para cálculos.	<code>char</code>	16 bits — caracteres Unicode.

TIPOS DE DADOS

Tipos primitivos

Primitivos		Linguagem Java	
Tipo	Descrição	Tipo	Capacidade de armazenamento
inteiro	Representa números inteiros positivos ou negativos.	byte	8 bits (de $-2^7 = -128$ a $2^7 - 1 = 127$)
		short	16 bits (de $-2^{15} = -32.768$ a $2^{15} - 1 = 32.767$)
		int	32 bits (de $-2^{31} = -2.147.483.648$ a $2^{31} - 1 = 2.147.483.647$)
		long	64 bits (de $-2^{63} = -9.223.372.036.854.775.808$ a $2^{63} - 1 = 9.223.372.036.854.775.807$)

TIPOS DE DADOS

Tipos primitivos

Primitivos		Linguagem Java	
Tipo	Descrição	Tipo	Capacidade de armazenamento
real	Representa números decimais positivos ou negativos. Também são chamados de <i>ponto flutuante</i> .	float	32 bits (de $1.40239846e - 45$ a $3.40282347e + 38$)
		double	64 bits (de $4.94065645841246544e - 324$ a $1.7976931348623157e + 308$)
lógico (booleano)	Representa apenas dois estados, verdadeiro (1) ou falso (0).	boolean	8 bits (assume dois valores: <code>true</code> ou <code>false</code>)

TIPOS DE DADOS

Tipos primitivos

- Na linguagem de programação Java, alguns tipos de dados têm as seguintes particularidades:
- O tipo long deve ser identificado com a letra “l” para não ser “compactado” pela linguagem em um tipo inteiro.
- A compactação ocorre como uma maneira de reduzir o uso de memória.

TIPOS DE DADOS

Tipos primitivos

- Por outro lado, a linguagem Java tenta utilizar o máximo de precisão possível.
- Dessa forma, se um elemento do tipo float (7 casas decimais após a vírgula) não for identificado com a letra “f”, será considerado do tipo double (15 casas decimais após a vírgula), o que pode gerar vários erros de compilação e execução.
- Estas são características da linguagem Java, não devendo ser estendidas a outras linguagens de programação.

TIPOS DE DADOS

Tipos primitivos

- Vejamos, a seguir, exemplos da declaração e atribuição de valores às variáveis, considerando o que foi mencionado.

```
int n = 4;
```

```
long numero = 4561;
```

```
float pi = 3.14f;
```

```
double tamanho = 3.873457486513793;
```


TIPOS DE DADOS

Tipos primitivos

- A linha `int n = 4` mostra como é uma declaração de uma variável em Java: consiste na definição de um tipo (`int`), de um nome para a variável (`n`), do símbolo de atribuição (`=`) e do valor que `n` irá assumir (`4`).

TIPOS DE DADOS

Tipos construídos

- Nos algoritmos, assim como nas linguagens de programação, existe a possibilidade de criarmos outros tipos de dados, chamados tipos construídos, obtidos pela combinação de alguns tipos, inclusive os primitivos.
- O tipo construído mais comum consiste na declaração de um conjunto de campos que compõe um registro. Por exemplo:

Algoritmo Exemplo_Registro

Var

Tipo reg_paciente = registro

nome: literal

idade: inteiro

peso: real

fim_registro

paciente: reg_paciente

...

TIPOS DE DADOS

Tipos construídos

- No Exemplo_Registro, o tipo reg_paciente foi construído com um conjunto de campos (variáveis) de diversos tipos de dados primitivos. Após a construção, é possível declarar variáveis que utilizem este tipo e, em nosso exemplo, criamos a variável paciente.

VARIÁVEIS

Introdução

- Uma variável pode ser entendida como uma posição de armazenamento que contém dados que podem ser modificados durante a execução do programa, podendo assumir qualquer um dos valores de um conjunto de valores.
- Nos algoritmos, as variáveis são utilizadas para representar valores desconhecidos, porém necessários para a resolução de um problema, podendo ser alterados, de acordo com a situação.
- Por isso, dizemos que as variáveis armazenam dados temporariamente.

VARIÁVEIS

Introdução

- Quando um algoritmo é transcrito para uma determinada linguagem de programação, as variáveis também terão a função de armazenar dados temporariamente na memória RAM do computador.
- Estes dados serão utilizados, durante o processamento, para a resolução do problema em questão

VARIÁVEIS

Identificação das variáveis

- Toda variável deve ser identificada, isto é, deve receber um nome ou identificador.
- O nome de uma variável deve ser único e estar de acordo com algumas regras:
- Não utilizar espaços entre as letras. Por exemplo, em vez de nome do cliente, o correto seria nome_do_cliente ou nomecliente.

VARIÁVEIS

Identificação das variáveis

- O caractere “sublinha” ou “underline”, representado por “_”, pode ser utilizado para substituir o espaço entre as letras.
- Não iniciar o nome da variável com algarismos (números). Por exemplo: não usar 2valor; o correto seria valor2.
- Não utilizar palavras reservadas, isto é, palavras que são utilizadas nos algoritmos para representar ações específicas. Por exemplo:

VARIÁVEIS

Identificação das variáveis

- Se: palavra que representa uma condição ou teste lógico;
- Var: palavra que representa a área de declaração de variáveis.
- Não utilizar caracteres especiais, como acentos, símbolos (?/:@# etc.), ç, entre outros.
- Ser conciso e utilizar nomes coerentes.

VARIÁVEIS

Identificadores de variáveis em Java

- Em Java, os nomes para as variáveis são case-sensitive, isto é, nomes com letras maiúsculas são diferenciados de nomes com letras minúsculas.
- Por exemplo: NomeCliente é diferente de nomecliente e também de nomeCliente.
- Nomes devem começar com uma letra, um caractere “sublinha” ou “underline” (_) ou o símbolo cifrão (\$).
- Os caracteres subsequentes podem também ser algarismos.

VARIÁVEIS

Identificadores de variáveis em Java

- Não utilizar caracteres especiais, como acentos, símbolos (?/:@# etc.), ç, entre outros, exceto os citados, anteriormente.
- As letras podem ser maiúsculas ou minúsculas.
- Não podem ser utilizadas palavras reservadas, como if, final, float, for, int etc.

CONSTANTES

Introdução

- Constantes são valores que não sofrem alterações ao longo do desenvolvimento do algoritmo ou da execução do programa.
- Por exemplo, na expressão a seguir, o valor 3.1415 é atribuído à constante pi e permanecerá fixo, até o final da execução.
 - $pi \leftarrow 3.1415$
 - $perimetro \leftarrow 2 * pi * raio$

CONSTANTES

Introdução

- Em Java, uma constante é uma variável declarada com o modificador final. Por exemplo:
- `final float pi = 3.1415f;`
- As constantes devem ser declaradas como variáveis, cujo valor atribuído permanecerá inalterado ao longo do programa.
- Por isso, são também chamadas de variáveis somente de leitura.

OPERADORES

Introdução

- Os operadores são utilizados para representar expressões de cálculo, comparação, condição e atribuição. Temos os seguintes tipos de operadores: de atribuição, aritméticos, relacionais e lógicos.

OPERADORES

Atribuição

- Os operadores de atribuição são utilizados para expressar o armazenamento de um valor, em uma variável.
- Este valor pode ser predefinido (variante ou não) ou ser o resultado de um processamento. Na notação algorítmica, o operador de atribuição pode ser representado por uma seta $\leftarrow$ ou pelo sinal “:=”.
- No nosso caso, adotaremos o primeiro, de acordo com o exemplo:

OPERADORES

Atribuição

nome ← "Fulano de tal"

resultado ← a + 5

valor ← 3.5

- Em Java, utiliza-se o sinal de igual "=", conforme exemplo.

nome = "Fulano de tal";

resultado = a + 5;

valor = 3.5;

OPERADORES

Aritméticos

- Os operadores aritméticos (Tabela 3.2) são utilizados para a realização dos diversos cálculos matemáticos.

OPERADORES

Aritméticos

Operador	Representação algorítmica	Notação para Java	Descrição para Java
Adição	+	+	Adiciona dois números ou variáveis: $a + b$.
Subtração	-	-	Subtrai dois números ou variáveis: $a - b$.
Multiplicação	*	*	Multiplica dois números ou variáveis: $a * b$.
Divisão	/	/	Divide dois números ou variáveis: a / b .
Módulo	mod	%	Retorna o resto da divisão de dois números ou variáveis: $a \% b$.
Incremento	$i = i + 1$	++	Adiciona 1 ao valor da variável: $a++$. Exemplo: na expressão $b = a++$, atribui o valor de a em b e depois incrementa a ; Na expressão $b = ++a$, atribui o valor de a em b após o incremento.

OPERADORES

Aritméticos

Operador	Representação algorítmica	Notação para Java	Descrição para Java
Decremento	$i = i - 1$	--	Subtrai 1 do valor da variável: a-- . Exemplo: na expressão b = a-- , atribui o valor de a em b e depois decrementa a ; Na expressão b = --a , atribui o valor de a em b após o decremento.
Adição e atribuição	$i = i + x$	+=	Adiciona e atribui o valor à variável: a += 3 .
Subtração e atribuição	$i = i - x$	-=	Subtrai e atribui o valor à variável: a -= 3 .
Multiplicação e atribuição	$i = i * x$	*=	Multiplica e atribui o valor à variável: a *= 3 .
Divisão e atribuição	$i = i / x$	/=	Divide e atribui o valor à variável: a /= 3 .
Módulo e atribuição	$i = i \bmod x$	%=	Calcula o resto da divisão e atribui à variável: a %= 3 .

OPERADORES

Aritméticos

- Nem todos os operadores aritméticos utilizados na realização de cálculos podem ser diretamente representados por símbolos computacionais.
- Alguns deles são representados por funções matemáticas, como no caso da exponenciação e da radiciação.
- Em Java, essas operações e algumas outras são realizadas utilizando métodos da classe `java.math`.

OPERADORES

Aritméticos

- Alguns destes métodos são mostrados no Apêndice B, Um pouco sobre o Java, disponível on-line na Sala Virtual.
- Funções matemáticas são programas especiais, existentes nas bibliotecas das linguagens de programação, executando cálculos mais complexos, não suportados pelos operadores matemáticos básicos.

OPERADORES

Relacionais

- Os operadores relacionais (Tabela 3.3) são utilizados para estabelecer uma relação de comparação entre valores ou expressões. O resultado desta comparação é sempre um valor lógico (booleano) verdadeiro ou falso.

OPERADORES

Relacionais

Operador	Representação algorítmica	Notação para Java	Descrição para Java
Maior que	>	>	a > b : se o valor de a for maior do que o de b, retorna verdadeiro, senão, falso.
Maior ou igual	>=	>=	a >= b : se o valor a de for maior ou igual ao de b, retorna verdadeiro, senão, falso.
Menor que	<	<	a < b : se o valor de a for menor que o de b, retorna verdadeiro, senão, falso.
Menor ou igual	<=	<=	a <= b : se o valor de a for menor ou igual ao de b, retorna verdadeiro, senão, falso.
Igual a	=	==	a == b : se o valor de a for igual ao de b, retorna verdadeiro, senão, falso.
Diferente de	<>	!=	a != b : se o valor de a for diferente de b, retorna verdadeiro, senão, falso.

OPERADORES

Lógicos

- Os operadores lógicos (Tabela 3.4) são utilizados para concatenar ou associar expressões que estabelecem uma relação de comparação entre valores. O resultado destas expressões é sempre um valor lógico, verdadeiro ou falso, uma vez que operam sobre valores booleanos.

Operador	Representação algorítmica	Notação para Java	Descrição para Java
E	.e.	&&	a && b: realiza a operação lógica E entre a e b.
OU	.ou.		a b: realiza a operação lógica OU entre a e b.
NÃO	.não.	!	! a: realiza a operação de negação em a.

OPERADORES

Lógicos

- Tabela verdade para os operadores lógicos

a	B	a .e. b (a && b)	a .ou. b (a b)	.não.a (!a)
falso	falso	falso	falso	verdadeiro
falso	verdadeiro	falso	verdadeiro	verdadeiro
verdadeiro	falso	falso	verdadeiro	falso
verdadeiro	verdadeiro	verdadeiro	verdadeiro	falso

OPERADORES

Lógicos

- Tabela verdade para os operadores lógicos

a	B	a .e. b (a && b)	a .ou. b (a b)	.não.a (!a)
falso	falso	falso	falso	verdadeiro
falso	verdadeiro	falso	verdadeiro	verdadeiro
verdadeiro	falso	falso	verdadeiro	falso
verdadeiro	verdadeiro	verdadeiro	verdadeiro	falso

OPERADORES

Precedência dos operadores

- Quando, em um algoritmo ou programa, uma expressão envolve mais de um operador, a ordem de avaliação segue a precedência dos operadores e, no caso das operações aritméticas, deve ser avaliada sempre a ordem **da esquerda para a direita**, quando calculam-se:
 - 1º) os fatoriais;
 - 2º) as funções, as potências e as raízes quadradas;
 - 3º) as multiplicações e divisões;
 - 4º) as adições e subtrações.

OPERADORES

Precedência dos operadores

- Exemplo:
 - $K \leftarrow A + B * \sqrt{16 - 10 \bmod 3}$ considere: $B \leftarrow 5$ e $A \leftarrow 2$
- Substituindo as variáveis pelos valores que lhe foram atribuídos, temos:
 - $K \leftarrow 2 + 5 * \sqrt{16 - 10 \bmod 3}$

OPERADORES

Precedência dos operadores

- 1º) As expressões $\sqrt{16}$ e $10 \bmod 3$ têm a mesma precedência, portanto, seguimos a ordem das expressões, considerando-as da esquerda para a direita.
- Então, calcula-se $\sqrt{16}$, obtendo-se o valor 4, e, em seguida, $10 \bmod 3$, resultando no valor 1. Reconstruindo-a com os valores obtidos, temos:

- $K \leftarrow 2 + 5 * 4 - 1$

OPERADORES

Precedência dos operadores

- 2º) Calculando-se $5 * 4 = 20$, temos:
 - $K \leftarrow 2 + 20 - 1$
- 3º) Todas as operações resultantes têm a mesma precedência e, desta maneira, seguimos calculando da esquerda para a direita:
 - $K \leftarrow 2 + 20 - 1$
 - $K \leftarrow 22 - 1$
 - $K \leftarrow 21$

OPERADORES

Precedência dos operadores

- Para impor a realização de uma operação ou de um conjunto de operações, em uma ordem diferente do padrão de precedência, podemos utilizar os parênteses ou colchetes.
- Dessa forma, a expressão anterior poderia ser escrita:
 - $K \leftarrow (A + B) * \sqrt{16 - 10 \bmod 3}$ considere: $B \leftarrow 5$ e $A \leftarrow 2$

OPERADORES

Precedência dos operadores

- Substituindo as variáveis pelos valores que lhe foram atribuídos, temos:
 - $K \leftarrow (2 + 5) * \sqrt{16} - 10 \bmod 3$
- 1º) Calculando-se a expressão entre parênteses $(2 + 5)$, temos o resultado:
 - $K \leftarrow 7 * \sqrt{16} - 10 \bmod 3$

OPERADORES

Precedência dos operadores

- 2º) As expressões $\sqrt{16}$ e $10 \bmod 3$ têm a mesma precedência, portanto, seguimos a ordem das expressões. Considerando-as da esquerda para a direita, calcula-se $\sqrt{16}$, obtém-se o valor 4 e, em seguida, $10 \bmod 3$ e obtém-se o valor 1. Reconstruindo-a com os valores obtidos, temos:
 - $K \leftarrow 7 * 4 - 1$

OPERADORES

Precedência dos operadores

- 3º) Calculando $7 * 4$, obtém-se 28, de modo que:
 - $K \leftarrow 28 - 1$
 - $K \leftarrow 27$
- Lembre-se de usar parênteses para isolar as operações que devem ser realizadas primeiro!

OPERADORES

Precedência dos operadores

- Apesar dos exemplos de precedência, com o uso de operadores aritméticos, devemos ter em mente que ela também se aplica a operações relacionais e lógicas.
- Logo, após a execução das operações aritméticas, devem ser realizadas as relacionais e, em seguida, as operações lógicas.
- A Tabela a seguir apresenta os operadores e a sua ordem de precedência.

Precedência de operadores na representação algorítmica.

Operador	Função realizada
(), []	Parênteses e colchetes são usados para agrupar expressões, determinando a precedência, a exemplo das expressões aritméticas.
^ ou **, √	Operador aritmético de potenciação e raiz quadrada.
*, /, mod	Operadores aritméticos de multiplicação, divisão e módulo.
+, -	Operadores aritméticos de adição e subtração.
=, >, <, >=, <=, <>	Operadores relacionais.
.não.	Operador lógico de negação.
.e.	Operador lógico e.
.ou.	Operador lógico ou.
←	Operador de atribuição.

OPERADORES

Precedência dos operadores

- Para exemplificar a precedência dos operadores relacionais e lógicos, vejamos o exemplo a seguir:
 - $A \leftarrow B + 2 > 5$.ou. $C <> 4$.e. $D = 0$ considerando: $B \leftarrow 5$; $C \leftarrow 3$; $D \leftarrow 1$
- Substituindo as variáveis pelos valores que lhe foram atribuídos:
 - $A \leftarrow 5 + 2 > 5$.ou. $3 <> 4$.e. $1 = 0$

OPERADORES

Precedência dos operadores

- 1º) Calculamos a expressão aritmética $5 + 2$:
 - $A \leftarrow 7 > 5 \text{ .ou. } 3 <> 4 \text{ .e. } 1 = 0$
- 2º) Avaliamos as operações relacionais: $7 > 5$ resulta .v.; $3 <> 4$ resulta .v. e $1 = 0$ resulta .f.
- Refazendo a expressão com os resultados:
 - $A \leftarrow \text{.f. .ou. .v. .e. .f.}$

OPERADORES

Precedência dos operadores

- 3º) Por último, avaliamos as operações lógicas. Lembre-se: o operador `.e.` tem prioridade sobre o operador `.ou.`, desta forma, avaliaremos primeiro a expressão `.v.` e `.f.`, cujo resultado é `.f.` Reescrevendo a expressão, temos:
 - $A \leftarrow .f. .ou. .f.$
 - $A \leftarrow .f.$

MUITO OBRIGADO!

Ficou com alguma
dúvida?



MSc. Jonathan Henrique Jeremias Souza
jonathansouza@libertas.edu.br
www.libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

Referências Bibliográficas

- SILVA, Maurício Samy. Fundamentos de HTML5 e CSS3. Novatec Editora, 2018.